

SANALLAŞTIRMANIN TEMELLERİ

[Kaynak](#)

İÇERİK

1. Sanallaştırmaya Genel Bakış (Virtualization Overview)
2. Sanallaştırma Bileşenleri (Virtualization Components)

Sanallaştırmaya Genel Bakış (Virtualization Overview)

Sanallaştırma kavramı ortaya çıkmadan önce, geleneksel BT (IT) altyapısındaki tartışmasız altın kural şuydu: **"Bir sunucu = Bir uygulama" (One server = one application)**.

Bilişim dünyasının ilk zamanlarında dijital hizmetler doğrudan fiziksel makineler (donanımlar) üzerinde çalıştırılıyordu. Kurulan her bir makinenin genellikle web sitesi barındırmak veya veri depolamak gibi tek ve net bir amacı vardı. Şirketler büyüyüp yeni hizmetler ekledikçe, doğal olarak fiziksel sunucuların sayısını da artırmak zorunda kalıyorlardı. "Her kutuya (sunucuya) tek bir iş" yaklaşımı, o dönemde güvenilir sistemler kurmanın standart ve tek yolu olarak kabul ediliyordu.

Uygulamada bunun anlamı şuydu: Bir şirket web sitesi, veritabanı, e-posta hizmeti ve şirket içi bir uygulama çalıştırmak istiyorsa, bu dört farklı iş için **dört ayrı fiziksel sunucu** satın almak zorundaydı. Fakat bu sistemin yarattığı devasa sorunlar çok geçmeden gün yüzüne çıktı.

Geleneksel "Tek Sunucu = Tek Uygulama" Modelinin Yarattığı Sorunlar

Bu eski yöntemin sürdürülemez olmasının altında yatan temel sorunları şu şekilde özetleyebiliriz:

- **Yüksek Maliyet (High Cost):** Birden fazla fiziksel sunucu satın almak başlı başına ciddi bir donanım masrafıdır. Ancak olay sadece cihazı almakla bitmez; bu sunucuların tükettiği elektrik, ürettikleri ısıyı soğutmak için gereken iklimlendirme sistemleri, donanım bakımları ve veri merkezinde (data center) kapladıkları fiziksel alanın kirası muazzam bir operasyonel maliyet yaratır.
- **Düşük Kullanım Oranı (Low Utilization - Kaynak İsrafı):** Belki de en can alıcı nokta burasıdır. Çoğu uygulama, üzerinde çalıştığı fiziksel sunucunun tam kapasitesine asla ihtiyaç duymaz. Sunucuların birçoğu sadece **%5 ile %20** arasında bir kapasiteyle çalışarak ömürlerini tüketiyordu. Bu durum; satın alınan **CPU (İşlemci)**, bellek (RAM) ve depolama kaynaklarının çok büyük bir kısmının tamamen israf edilmesi anlamına geliyordu.

- **Yavaş Kurulum ve Dağıtım (Slow Deployment):** Sisteme yeni bir fiziksel sunucu eklemek ve ayağa kaldırmak (sipariş etmek, kablolamak, işletim sistemi kurmak, yapılandırmak) günler, hatta bazen haftalar sürüyordu. Çeviklik (agility) sıfıra yakındı.
- **Ölçeklendirme Zorluğu (Hard to Scale):** Çalışan bir uygulamanın aniden daha fazla kaynağa (örneğin trafik arttığı için daha fazla RAM'e) ihtiyacı olduğunda, mevcut sunucu yetmiyorsa gidip yeni bir fiziksel sunucu daha satın almaktan başka pratik bir çare yoktu.

Kısacası şirketler, potansiyelinin çok küçük bir kısmını kullandıkları devasa donanım yığınlarına servet ödüyorlardı.

Donanımı Güvenli ve Verimli Bir Şekilde Paylaşma İhtiyacı

Tüm bu maliyet ve hantallık krizinin ortasında, sanallaştırma teknolojisi oyunu değiştiren şu soruyu sordu: **"Peki ya birden fazla uygulama, aynı fiziksel sunucuyu GÜVENLİ bir şekilde paylaşabilseydi?"**

İşte bu noktada sisteme **Hypervisor (Hipervizör)** adını verdiğimiz bir sanallaştırma katmanı dahil oldu.

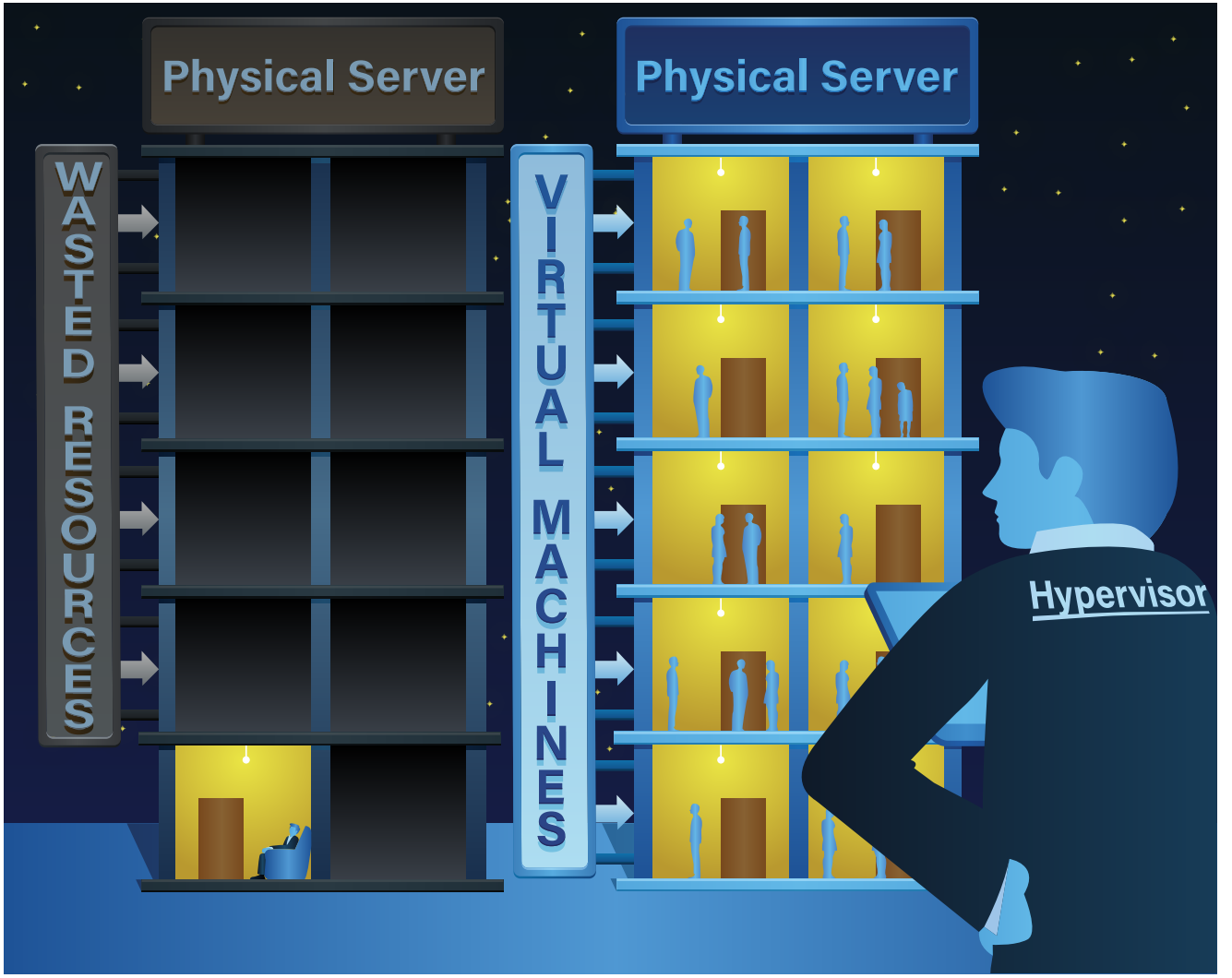
- **Hypervisor nedir?** Fiziksel donanım ile sanal makineler arasında çalışan, bir nevi "hakem" görevi gören özel bir yazılım katmanıdır. Görevi, donanım kaynaklarını (CPU, RAM vb.) sanal bilgisayarlara paylaşmak ve her bir sanal bilgisayarın sanki tek başına fiziksel bir bilgisayarmış gibi bağımsız davranmasını sağlamaktır.

Kavramı Oturtmak İçin: Bina Analojisi

Sanallaştırma mantığını aylar sonra bile unutmamak için şu harika bina benzetmesini aklımıza kazıyoruz.

Eski Yöntem (Sanallaştırma Öncesi): Sadece tek bir kişinin 10 katlı devasa bir binada tek başına yaşadığını hayal edin.

- Bu kişi aslında binanın sadece tek bir katını kullanıyor. Ancak tüm binanın elektrik, su, temizlik ve güvenlik masraflarını tek başına karşılamak zorunda.
- Binanın geri kalan 9 katı tamamen boş, atıl durumda ve israf ediliyor.
- Bu durum, o kişinin ihtiyaçları göz önüne alındığında son derece pahalı, verimsiz ve gereksizdir. *(İşte bu %10 kapasiteyle çalışan fiziksel sunucudur).*



****Yeni Yöntem (Sanallaştırma Sonrası):**** Şimdi aynı 10 katlı binanın ayrı ayrı dairelere (apartman mantığı) bölündüğünü düşünün.

- Her dairenin kendi kapısı, kendi duvarları, mutfuğı ve mahremiyeti (privacy) vardır.
- Farklı insanlar (uygulamalar), birbirlerinin alanına müdahale etmeden, birbirlerini rahatsız etmeden bağımsız bir şekilde yaşayabilirler.
- Ancak hepsi binanın ana yapısını (temelini, asansörünü, ana su ve elektrik hattını) **ortaklaşa** kullanır. Bu da yaşamı herkes için çok daha ucuz ve verimli hale getirir.

Analojinin Teknik Terimlerle Eşleşmesi (Kritik Nokta)

Bu benzetmeyi BT dünyasına tam olarak şöyle çeviriyoruz:

- **Bina:** Fiziksel Sunucu (Donanım)
- **Daireler:** **Virtual Machine - VM** (Sanal Makineler)
- **Kiracılar:** Uygulamalar veya İşletim Sistemleri
- **Bina Yöneticisi:** **Hypervisor** (Binayı güvenli bir şekilde dairelere bölen ve kaynakları adil dağıtan yazılım)

Sonuç: Bu mimari sayesinde oluşturulan her bir sanal bilgisayar, yani **Virtual Machine (VM)**; altında yatan aynı fiziksel donanımı (işlemciyi, disk) diğer VM'lerle paylaşmasına

rağmen, tamamen **bağımsız bir sistem** gibi davranır. Her VM'in kendine ait, diğerlerinden izole edilmiş bir işletim sistemi, kendi uygulamaları ve kendi ağ ayarları vardır. Biri çökerse veya hacklenirse, yan dairedeki (diğer VM) etkilenmez.

Sanallaştırma Bileşenleri (Virtualization Components)

Bir önceki notta sanallaştırmanın mantığını "10 katlı bina" analojisiyle oturtmuştuk. Şimdi bu binanın nasıl inşa edildiğine, yöneticisinin kim olduğuna ve odaların nasıl bölündüğüne teknik açıdan bakıyoruz.

1. Hypervisor (Bina Yöneticisi)

Hypervisor (Hipervizör), sanallaştırmanın kalbi ve temel teknolojisidir. Basitçe, sanal makineleri (VM) yaratan ve yöneten özel bir yazılımdır. Sistemin "hakemi" veya "bina yöneticisi" olarak düşünebiliriz.

Bir Hypervisor tam olarak ne yapar?

- Fiziksel bilgisayarı donanımsal olarak parçalara böler.
- Her bir **Virtual Machine (VM - Sanal Makine)** için işlemci (**CPU**), bellek (**RAM**) ve depolama (Storage) kaynaklarını adil bir şekilde paylaştırır.
- İzolasyon (Isolation)** sağlar: Sanal makineleri birbirinden kesin çizgilerle ayırır. Biri çökerse, diğerleri güvenli çalışmaya devam eder.
- VM'lerin **Yaşam Döngüsünü (Lifecycle)** yönetir: Sanal makineleri başlatma, durdurma, duraklatma (pause), birebir kopyalama (clone) ve silme yetkisine sahiptir.

Hypervisor Tipleri (Mimari Farklılıklar)

Hypervisor'ler, evdeki laboratuvarımızdan devasa veri merkezlerine (Data Center) kadar farklı kullanım amaçlarına göre iki ana mimariye ayrılır:

- Type 1 (Bare-Metal / Doğrudan Donanım):** Bu tipte, Hypervisor doğrudan fiziksel sunucunun donanımı üzerine kurulur. Arada Windows veya Linux gibi bir "Ana İşletim Sistemi" (Host OS) yoktur. İşletim sistemi yükü olmadığı için inanılmaz hızlı ve verimlidir. Üretim ortamlarında (Production) ve devasa sunucularda her zaman bu kullanılır.
- Type 2 (Hosted / İşletim Sistemi Üzerinde):** Bizim bilgisayarlarımızda kullandığımız modeldir. Mevcut bir işletim sisteminin (Örn: Windows 11) üzerine normal bir program gibi kurulur. Kurulumu çok kolaydır. Sızma testleri, yeni şeyler öğrenmek (Learning) veya yazılım testleri için mükemmeldir. Ancak Type 1 kadar performanslı değildir çünkü donanıma giderken ana işletim sisteminden geçmek zorundadır.

Kullanım Senaryoları Tablosu:

Aşağıdaki tablo, hangi iş için hangi tipin daha uygun olduğunu gösteriyor. (Not: Teknik olarak birini diğerinde yapabilirsiniz ama performans ve güvenlik açısından "en doğru yol" budur).

Kullanım Senaryosu (Use Case)	Type 1 (Bare-Metal)	Type 2 (Hosted)
Test Malicious Files (Zararlı Yazılım Analizi)		X
Production Server (Canlı Üretim Sunucusu)	X	
Database Server (Veritabanı Sunucusu)	X	
Software Testing (Yazılım Testleri)		X
Kali Linux (Sızma Testi İşletim Sistemi)		X
Data Center (Veri Merkezi)	X	

Zararlı Yazılım (Malware) Analizi İçin Kritik Not: > Bir virüsü veya zararlı dosyayı analiz ederken en büyük risk, virüsün sanal makineden kaçıp kendi gerçek bilgisayarımıza (Host makineye) bulaşmasıdır. Bunu engellemek için Guest (Sanal) makine ile Host (Ana) makinenin ağ iletişimini (Network) tamamen keserek izole etmeliyiz. Ayrıca, Host ve Guest makinelerde **farklı işletim sistemleri** (Örn: Host macOS, Guest Windows) kullanmak, virüsün ana makinede çalışma ihtimalini sıfıra yaklaştırarak ekstra bir güvenlik katmanı sağlar.

2. Virtual Machines (VM - Sanal Makineler / Daireler)

Virtual Machine (VM), Hypervisor tarafından yaratılan sanal bir bilgisayardır. Analojimizdeki apartman dairelerine denk gelir.

Sanal olmasına rağmen, sistem tamamen **gerçek bir makine gibi** davranır çünkü:

- Kendi sanal CPU'su, RAM'i, depolama alanı ve kendi ağ kartı (Network Interface) vardır.
- Üzerine istediğiniz işletim sistemini kurabilirsiniz (Windows, Linux, macOS).
- Diğer VM'lerden tamamen izole edilmiştir.

Siber Güvenlikte VM Neden Şarttır?

1. **İhtiyaç (Kali Linux):** CTF (Capture The Flag) çözeceksiniz veya sızma testi yapacaksınız. Tüm araçların kurulu olduğu **Kali Linux**'a ihtiyacınız var. Mevcut Windows'unuzu silip Linux kurmak yerine, bilgisayarınıza bir Type 2 Hypervisor (**Oracle VirtualBox** veya **VMware Workstation**) indirip, içine Kali Linux'u bir VM olarak saniyeler içinde kurabilirsiniz.
2. **Güvenlik (İzolasyon):** Şüpheli bir .exe indirdiniz. Kendi sisteminizde açıp riske girmek yerine, bu dosyayı ağ bağlantısı kesilmiş, izole bir Windows VM içinde açıp sistemde ne gibi değişiklikler yaptığını güvenle izleyebilirsiniz. İşiniz bitince VM'i siler geçersiniz.

3. Containers (Konteynerler / Daire İçindeki Odalar)

Konteyner mimarisi, sanallaştırmanın evrim geçirmiş, daha modern ve hafif (lightweight) halidir. Bir **Container**, sadece tek bir uygulamayı ve o uygulamanın çalışması için gereken kütüphaneleri barındıran izole bir kutudur.

Konteynerin VM'den En Büyük Farkı Nedir? (MANTIK ZİNCİRİ)

Bir VM kurduğunuzda, uygulamanın altına koca bir Windows veya Linux işletim sistemi kurmak zorundasınızdır (Bu GB'larca yer ve ciddi RAM tüketimi demektir).

Ancak Konteynerler **yeni bir işletim sistemi kurmazlar**. Bunun yerine, üzerinde çalıştıkları ana bilgisayarın (Host) **Kernel'ini (Çekirdeğini)** ödünç alırlar.

(Kernel: İşletim sisteminin donanımla, bellek yönetimiyle ve programların çalışmasıyla ilgilenen kalbidir).

Aşağıdaki şematik yapı, bu farkı zihne kazımak içindir:

Plaintext

[VM MİMARİSİ]	[CONTAINER MİMARİSİ]
-----	-----
Uygulama A	Uygulama A
Binaries/Libs	Binaries/Libs
Guest OS (Ağır yük)	-----
-----	Uygulama B
Hypervisor	Binaries/Libs
-----	-----
Host İşletim Sistemi	Container Motoru (Örn: Docker)
-----	-----
Fiziksel Donanım	Host İşletim Sistemi (Ortak
Kernel)	-----
-----	-----
-	Fiziksel Donanım

-	

Yukarıdaki şemada görüldüğü gibi, VM her uygulama için koca bir Guest OS (Sanal İşletim Sistemi) kurarken, Konteyner alttaki mevcut Kernel'i ortak kullanır.

Konteynerlerin Özellikleri ve Kuralları:

- **İnanılmaz Hızlıdır:** Kernel ortak olduğu ve işletim sistemi boot edilmediği için konteynerler milisaniyeler içinde başlar.
- **Minimum Kaynak Tüketir:** Geleneksel VM'lere göre çok daha az CPU ve RAM kullanır.
- **Platform Bağımsızlık:** Uygulama ve tüm bağımlılıkları tek pakette olduğu için, "benim bilgisayarımda çalışıyordu, sunucuda çöktü" problemi yaşanmaz. Geliştirme ve test

süreçleri için kusursuzdur.

- **Kritik Kısıtlama (Kernel Bağımlılığı):** Konteynerler host makinenin Kernel'ini paylaştığı için **aynı tip işletim sistemine** sahip olmalıdır. Yani, alt yapısında Linux Kernel'i çalışan bir ana makinede, **Windows Konteyneri ÇALIŞTIRAMAZSINIZ**. (Çünkü Windows'a özgü sistem çağrılarını Linux kernel'i anlayamaz).
- **İzolasyon:** Bir konteynerdeki uygulama çökse veya sızılrsa bile, diğer konteynerler bu durumdan etkilenmez.

Araç (Tool): Docker

Bir sanal makinenin içinde (veya direkt fiziksel makinede) konteyner oluşturmanın ve yönetmenin açık ara en kolay, endüstri standardı yolu **Docker** kullanmaktır. Docker, "Containerization" (Konteynerleştirme) sürecini basitleştiren açık kaynaklı bir platformdur.

💡 Akılda Kalması İçin Son Özet:

- **Virtual Machines (VM):** Maksimum izolasyon, maksimum esneklik sağlayan "**Tam Teşekküllü Dairelerdir**". İçine ne isterseniz kurarsınız ama hantaldır.
- **Containers:** Hızlı kurulan, çok az kaynak tüketen, anında ölçeklenebilen "**Daire İçindeki Odalardır**". Geliştiricilerin (DevOps) en yakın dostudur.